

Capstone Project: Titanic Survivability Classification (MLOps Pipeline)

1. Introduction

This project demonstrates a complete MLOps pipeline for the Titanic dataset, focusing on binary classification (Survived: Yes/No). The goal is not only to build a predictive model but also to implement modern MLOps practices including data versioning, experiment tracking, API deployment, CI/CD, logging, and monitoring. This provides hands-on experience for real-world machine learning workflows.

2. Project Overview & Architecture

Dataset: Titanic dataset (train.csv with 891 samples). Features include passenger demographics, fare, class, family members, and more.

Task: Binary classification – predicting survival (0 = Not Survived, 1 = Survived).

Tools Used:

- Git + GitHub (version control)
- DVC (data versioning, optional)
- MLflow (experiment tracking & model registry)
- FastAPI + Pydantic (API service)
- Docker (containerization)
- GitHub Actions (CI/CD)
- Python logging + SQLite (logging and persistence)
- Prometheus + Grafana (optional monitoring)

Architecture highlights:

- Data and code versioning using Git/DVC.
- MLflow server to track experiments and register models.
- FastAPI service for inference with input validation.
- Docker container for reproducibility and deployment.
- GitHub Actions workflow for automated testing, building, and deployment.
- Logging and monitoring for reliability.

3. Part 1: Repository & Data Versioning

The repository is structured to separate raw data, source code, API service, and models. The data preparation script handles missing values (e.g., filling or flagging missing Age and

Cabin), creates engineered features (such as IsChild, FamilySize), encodes categorical variables, and saves a cleaned dataset.

Data versioning can be managed with DVC, ensuring reproducibility while keeping large files out of Git.

4. Part 2: Model Development & Experiment Tracking

Several machine learning models (Logistic Regression, Random Forest, Gradient Boosting) are trained and compared. MLflow is used to log parameters, training metrics (accuracy, precision, recall, F1), and feature importance. MLflow enables experiment comparison and model registry to promote the best model.

5. Part 3: API & Docker Packaging

The API is built using FastAPI, with input validation provided by Pydantic schemas. It exposes two endpoints:

- GET /health: Returns system status.
- POST /predict: Accepts passenger features in JSON format and returns survival prediction and probability.

The API is containerized with Docker, ensuring environment consistency across systems.

6. Part 4: CI/CD with GitHub Actions

A GitHub Actions workflow (ci.yml) ensures continuous integration and deployment. The workflow includes linting, unit testing, Docker image building, and optional deployment. Docker images are pushed to Docker Hub using stored secrets for secure authentication.

7. Part 5: Logging & Monitoring

Every API request is logged with features, timestamp, prediction, probability, and latency. Logs are rotated daily and stored both in files and in a SQLite database. Optional monitoring with Prometheus and Grafana can track metrics such as request volume, latency, error rates, and prediction distribution.

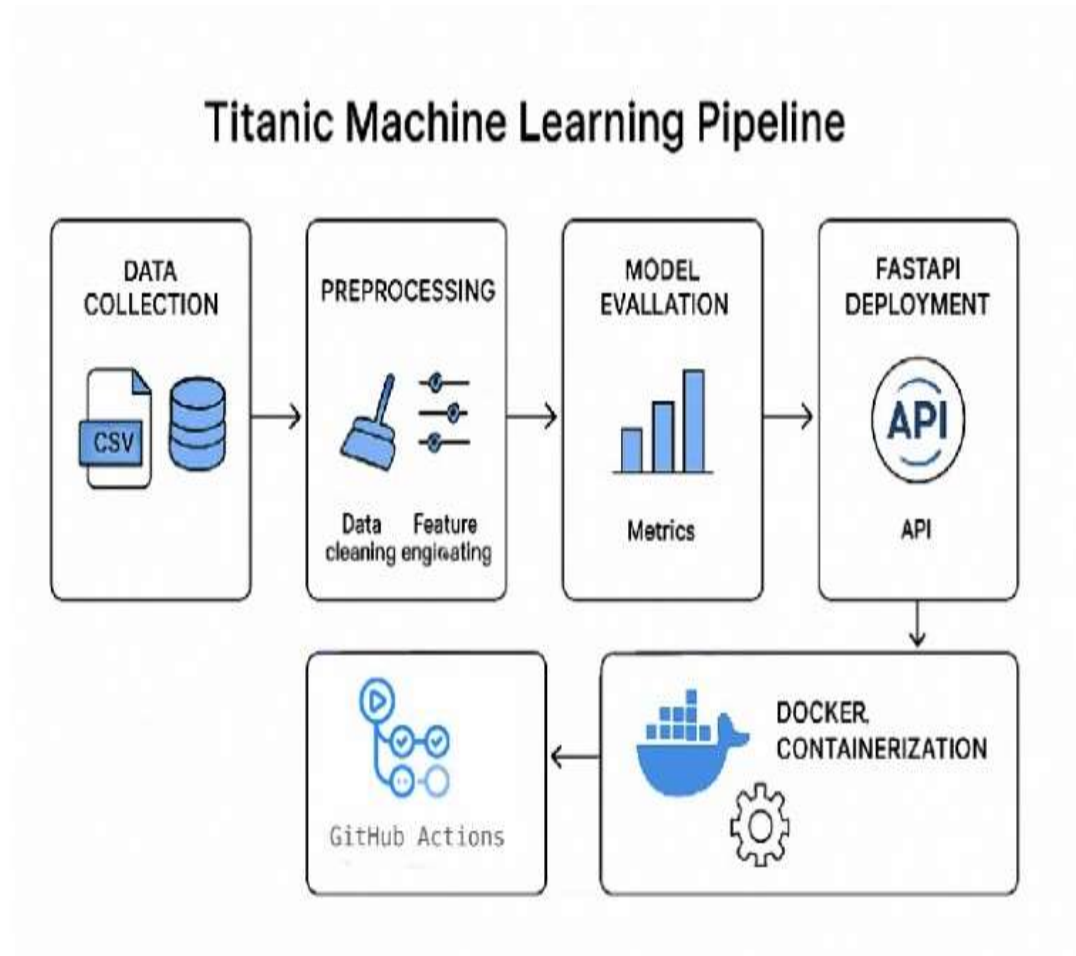
8. Part 6: Summary & Demo

Deliverables:

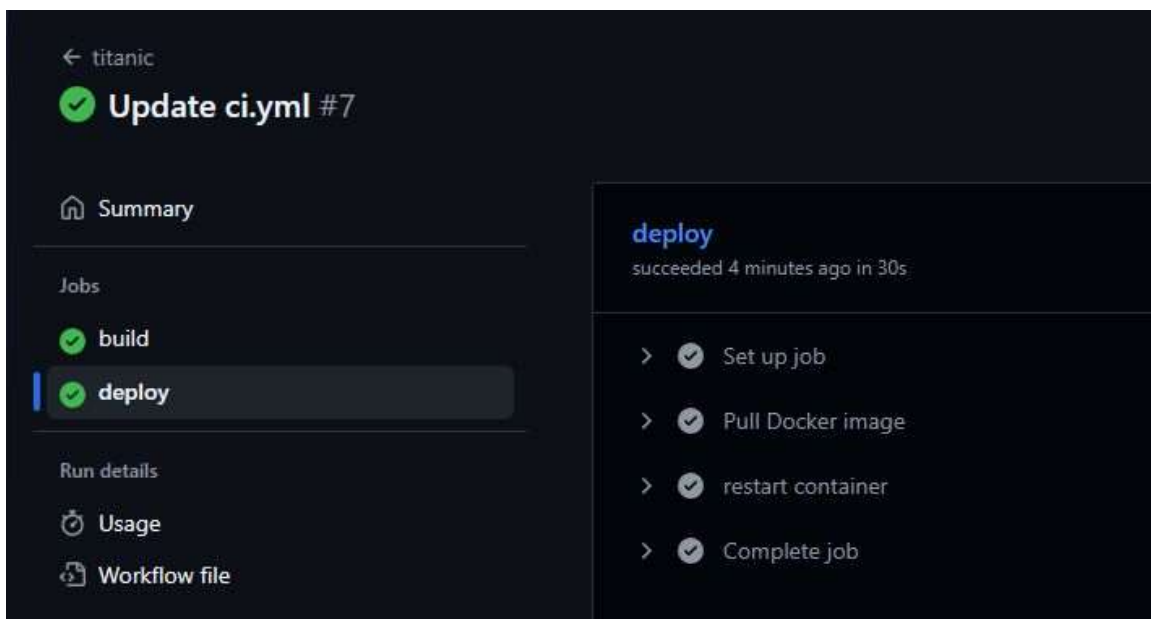
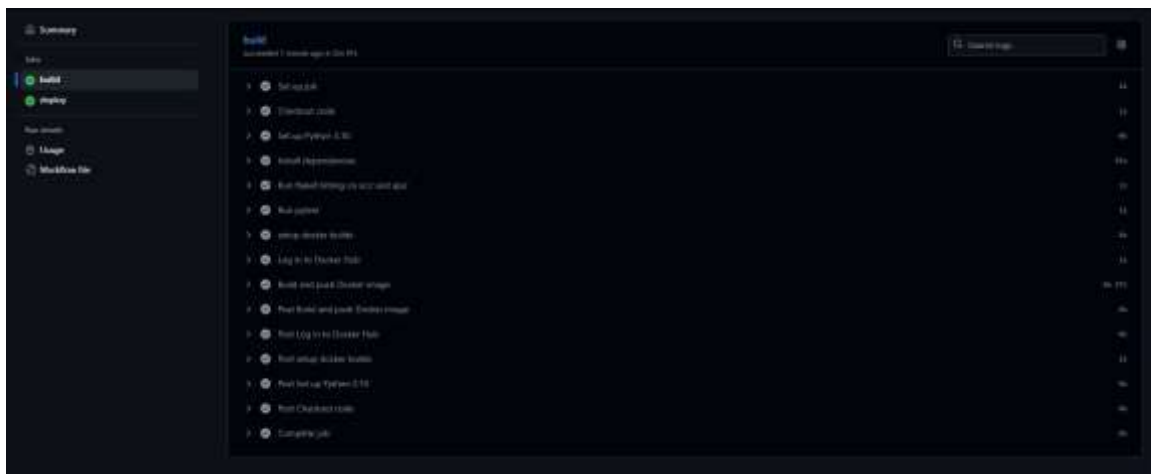
- Professional GitHub repository with code and configurations.
- Docker Hub image for the API server.
- MLflow experiment tracking with comparison screenshots.
- CI/CD workflow demonstration via GitHub Actions.
- Monitoring dashboards (optional).

This project demonstrates the end-to-end lifecycle of a machine learning project, integrating MLOps best practices for data, model, deployment, and monitoring.

Architecture diagram:



CI/CD workflow snapshot:



- MLflow comparison screenshot

Experiments

Titanic_Survival_Experiments [Provide Feedback](#) [Add Description](#)

Runs Models **Experimental** Evaluation Traces

metrics.rmse < 1 and params.model = "tree" Time created State: Active Columns

Group by

	Run Name	Created	Dataset	Duration	Source	Models	Metrics
							accuracy $\uparrow$
	RandomForest	17 minutes ago	-	6.5s	trainpy	RandomForest_model	0.981741573...
	RandomForest	20 minutes ago	-	6.5s	trainpy	model	0.981741573...
	GradientBoosting	17 minutes ago	-	6.4s	trainpy	GradientBoosting_model	0.910812359...
	GradientBoosting	20 minutes ago	-	6.7s	trainpy	model	0.910812359...
	LogisticRegression	17 minutes ago	-	8.6s	trainpy	LogisticRegression_model	0.807584269...
	LogisticRegression	20 minutes ago	-	20.7s	trainpy	model	0.807584269...